

TECHNICAL SPECIFICATION

PhishGuard

Technical Stack & Architecture Report

A comprehensive breakdown of every framework, library, and technical specialty used across the frontend, backend, detection pipeline, and automation layer of the PhishGuard explainable phishing-detection platform.

React + Vite

FastAPI

Supabase

LLM Agent Layer

n8n Automation

Vercel

Table of Contents

01	Project Overview	3
02	System Architecture at a Glance	3
03	Frontend — Frameworks & Libraries	4
04	Backend — Frameworks & Libraries	5
05	Detection Pipeline — Layers 0 to 4	6
06	Data, Auth & Security Layer	7
07	Automation Layer — n8n Workflows	7
08	Deployment & Infrastructure	8
09	Supporting Tools & External Resources	8
10	Complete Stack — Quick Reference Table	9

SECTION 01

Project Overview

PhishGuard is an explainable, layered phishing-detection platform built for a hackathon prototype but architected in a production-shaped way. A user submits a suspicious email, URL, screenshot, or QR code; the system runs it through a deterministic rule-based pipeline first, then a configurable LLM agent synthesizes a plain-English explanation and, for high-risk results, generates incident reports and network-containment artifacts automatically.

The system is split into four cooperating parts: a **React frontend** for the dashboard and scan interface, a **FastAPI backend** running the detection pipeline and LLM orchestration, **Supabase** for authentication and data storage, and an optional **n8n automation layer** handling inbox ingestion and SOAR-style incident response.

Design principle: the LLM is the explanation and orchestration layer, never the sole decision-maker. Every risk score traces back to deterministic, rule-based evidence — this is what separates PhishGuard from a simple "ask an AI if it's a scam" wrapper.

02 · System Architecture at a Glance

Presentation

React (Vite) dashboard + scan UI, deployed on Vercel, styled with Tailwind CSS and a dark/lime design system.

Application Logic

FastAPI backend running the 5-layer detection pipeline and LLM agent orchestration, deployed on Render/Railway.

Data & Identity

Supabase Postgres + Supabase Auth with Row Level Security, storing scans, evidence, connections, and reports.

Frontend — Frameworks & Libraries

The frontend is a single React application covering the public landing page, authentication, dashboard, scan submission, evidence-card results, connections management, and agent-profile settings — all sharing one design system and navigation shell.

Library / Tool	Category	Role in PhishGuard
React (Vite)	Core framework	Component-based UI; Vite provides fast dev server and optimized production builds.
Tailwind CSS	Styling	Utility-first styling; hosts the custom black + neon-green design tokens as theme variables.
React Router	Routing	Client-side routing across landing, auth, dashboard, scan, connections, and settings pages.
@supabase/supabase-js	Auth & data client	Handles sign-in/sign-up, session tokens, and optional Supabase Realtime subscriptions.
Recharts	Data visualization	Renders the risk-score ring/gauge and the 30-day scan-trend line chart on the dashboard.
Framer Motion	Animation	Animates Evidence Cards sliding/fading in and the 4-step live scan-progress indicator.
lucide-react	Iconography	Thin-stroke line icons for shield, alert, link, mail, and QR indicators across the UI.
shadcn/ui	Component primitives	Optional accessible base components (dialogs, tabs, toasts) layered under the custom theme.

Design system summary

Base #0A0A0A

Surface #131313

Accent #C8FF33

Risk-High #E5484D

Risk-Medium #F5C244

Mono-accent typography

Outlined card panels

Backend — Frameworks & Libraries

The backend is an asynchronous Python service exposing REST and WebSocket endpoints, running the five-layer detection pipeline, orchestrating the LLM explanation agent, and managing Gmail/app connections.

Library / Tool	Category	Role in PhishGuard
FastAPI	Core framework	Async REST + WebSocket API; serves the scan, reports, connections, and agent-profile endpoints.
Uvicorn	ASGI server	Production-grade async server process running the FastAPI app.
supabase-py	Data client	Server-side Supabase client using the service role key for privileged reads/writes.
httpx	Async HTTP client	Unshortens URLs by following redirects (depth-capped) and fetches remote content asynchronously.
tldextract	URL parsing	Decomposes URLs into subdomain / registrable domain / TLD for typosquatting and subdomain-spoofing checks.
python-Levenshtein	String similarity	Calculates edit distance against known brand domains to catch near-identical typosquats.
idna	Encoding	Converts internationalized domain names to Punycode to expose homograph/IDN spoofing attempts.
pyzbar + opencv-python	Computer vision	Decodes QR code payloads from uploaded images for quishing detection.
beautifulsoup4	HTML parsing	Strips and inspects HTML email bodies, separating visible text from hidden/obfuscated content.
LangChain / LiteLLM	LLM orchestration	Provider-agnostic interface to the explanation/SOAR agent; keeps the LLM provider swappable.
Celery + Redis	Background jobs	Queues longer-running scan and report-generation tasks outside the request/response cycle.
Pydantic	Validation	Strict schema validation for every API request/response and for the JSON evidence bundle passed to the LLM.

SECTION 05

Detection Pipeline — Layers 0 to 4

Each submission passes through five discrete, independently inspectable layers. Only the final layer touches a language model, and it receives strictly validated JSON — never raw untrusted text.

LAYER 0 Normalization & Ingestion

Unshortens URLs, decodes QR payloads, strips HTML, isolates domains, converts IDN to Punycode.

LAYER 1 Deterministic Engine

Pure Python/regex rules: Levenshtein brand matching, high-risk TLDs, subdomain spoofing, open-redirect detection.

LAYER 2 NLP & Behavioral Extraction

Detects urgency, fear, and authority-impersonation language; extracts exact token spans for UI highlighting.

LAYER 3 Risk Aggregator & Scorer

Deterministic, unit-testable weighted scoring — no LLM involvement — producing a 0-100 composite score.

LAYER 4 Generative Explainability & SOAR

LLM agent explains the verdict in plain English and, above threshold, drafts incident reports and containment rules.

Why layered: this structure is what lets the project demonstrate deterministic, measurable precision/recall rather than an opaque "trust the model" verdict — a common weakness judges specifically look to penalize in AI-security hackathon entries.

Data, Auth & Security Layer

Component	Role
Supabase Auth	Email/password and Google OAuth sign-in; issues JWTs consumed by the FastAPI backend on every request.
Row Level Security (RLS)	Postgres policies ensuring each user can only read their own scans, evidence, connections, and reports.
Encrypted OAuth tokens	Gmail (and future provider) refresh tokens stored encrypted, never in plaintext.
Prompt-injection guardrail	Untrusted message content is summarized into structured evidence before it ever reaches the LLM, preventing attacker-controlled text from overriding agent instructions.

07 · Automation Layer — n8n Workflows

n8n complements the backend as the ingestion and SOAR-automation layer, without duplicating the actual detection logic:

Node / Stage	Function
Gmail Trigger node	Watches a connected inbox and forwards new messages into the pipeline.
HTTP Request node	Calls the backend's <code>/api/scans</code> endpoint; n8n is the courier, not the decision-maker.
IF node	Branches on returned severity (LOW / MEDIUM / HIGH).
Slack / Discord / SMTP / Sheets nodes	Fan out HIGH-risk actions: team alerts, CERT-style report emails, and incident logging.

Deployment & Infrastructure

Frontend

Vercel — React/Vite static + edge deployment, automatic preview URLs per commit.

Backend

Render / Railway / Fly.io — chosen over Vercel's Python runtime to support long-running WebSockets and background jobs.

Data

Supabase — managed Postgres, Auth, and optional Realtime channels, hosted separately from app compute.

09 · Supporting Tools & External Resources

Tool	Purpose	Why it's useful here
Google Safe Browsing API	Threat intel	Free-tier blocklist check to reinforce Layer 1 without building a custom domain database.
VirusTotal API	Threat intel	Secondary reputation signal for submitted URLs and files.
Nylas / Composio	Connector platform	Pre-built OAuth connectors for Gmail/Outlook/Slack, reducing custom integration code.
Sentry	Error tracking	Live error visibility during development and the hackathon demo itself.
Upstash Redis	Rate limiting	Serverless-friendly request throttling for the public scan endpoint.
n8n	Workflow automation	Visual ingestion + SOAR automation layer, detailed in Section 07.

Complete Stack — Quick Reference Table

Layer	Frameworks / Libraries	Purpose
Frontend	React, Vite, Tailwind CSS, React Router, Recharts, Framer Motion, lucide-react, shadcn/ui	UI, routing, charts, animation, icons
Auth & Data	Supabase (Postgres, Auth, RLS, Realtime), supabase-js, supabase-py	Identity, storage, access control
Backend Core	FastAPI, Uvicorn, Pydantic	API surface, validation, async serving
Detection Engine	tlextract, python-Levenshtein, idna, pyzbar, opencv-python, beautifulsoup4, httpx	Deterministic Layers 0–1 rule checks
LLM Layer	LangChain / LiteLLM	Layer 4 explanation & SOAR generation
Background Processing	Celery, Redis	Async scan/report jobs
Automation	n8n	Ingestion + SOAR workflow orchestration
Deployment	Vercel, Render / Railway / Fly.io, Supabase Cloud	Hosting for frontend, backend, and data respectively
Observability	Sentry, Upstash Redis	Error tracking, rate limiting
Threat Intel	Google Safe Browsing API, VirusTotal API	External reputation signals

This document is a technical reference compiled for the PhishGuard hackathon prototype. It reflects the architecture and tooling decisions made during planning and is intended to accompany the project's backend and frontend build prompts.